

Use of Large Language Models for Intent-Based Configuration in Software-Defined Networks

Derick G. Andrighetti¹

¹Institute of Computing – University of Campinas (Unicamp)
São Paulo, Brazil.

²Artificial Intelligence / Machine Learning

derickcollege@gmail.com

Abstract. *Software-Defined Networking (SDN) and Intent-Driven Networking (IDN) enable programmable management by separating control and data planes and defining policies as intents. In ONOS, intents represent configurations such as host connectivity but require manual specification. This project develops and evaluates a mediator that uses a Large Language Model (LLM) to interpret natural-language requests and translate them into ONOS intents with verification. Experiments in a Mininet testbed using linear and tree topologies measure translation success rates, latency decomposition, and error distribution. The study analyzes the feasibility of using LLMs as cognitive interfaces for intent-based configuration, highlighting the trade-off between semantic interpretation capabilities and inference latency.*

Resumo. *As Redes Definidas por Software (SDN) e as Redes Dirigidas por Intenções (IDN) permitem o gerenciamento programável por meio da separação entre planos e da definição de políticas como intents. No ONOS, intents representam configurações como conectividade entre hosts, mas exigem especificação manual. Este projeto desenvolve e avalia um mediador que utiliza um Modelo de Linguagem de Grande Escala (LLM) para interpretar solicitações em linguagem natural e traduzi-las em intents do ONOS com verificação. Experimentos em um testbed Mininet com topologias lineares e em árvore medem taxas de sucesso, decomposição de latência e distribuição de erros. O estudo avalia a viabilidade do uso de LLMs como interfaces cognitivas para configuração orientada por intenções em SDN, destacando o impacto da latência de inferência na aplicabilidade em tempo real.*

1. Introduction

Software-Defined Networking (SDN) facilitates network management by centralizing control logic and offering programmable interfaces. This architecture allows operators to define forwarding policies through software rather than hardware configuration [Kreutz et al. 2015]. Building on this foundation, Intent-Driven Networking (IDN) aims to abstract the complexity of these policies. In an IDN model, the operator declares *what* the network should do, and the system determines *how* to implement it [Minhas et al. 2024a].

Despite these advances, a semantic gap remains in practical implementations. Controllers like ONOS¹ provide an intent framework, but these intents are structured software objects that require precise parameterization. There is no native mechanism to translate high-level natural language instructions into these formal specifications. Consequently, network configuration often relies on manual scripting or specialized knowledge of the controller API.

Large Language Models (LLMs) offer a potential solution to bridge this gap. Their ability to interpret code and natural language suggests they could act as mediators between human operators and SDN controllers. However, applying probabilistic models to deterministic network control introduces challenges regarding reliability, latency, and state consistency. An incorrect interpretation could disrupt connectivity, while excessive inference time could make the system unresponsive.

This project develops and evaluates an LLM-based mediator designed to translate natural language instructions into ONOS intents. The system implements a translation pipeline that includes interpretation, JSON formatting, API execution, and a crucial verification step to ensure the network state reflects the user request.

The study assesses the viability of this architecture through a quantitative analysis of success rates and latency components. Experiments were conducted using Mininet² to emulate linear and tree topologies controlled by ONOS. The results quantify the trade-off between the ease of natural language configuration and the performance overhead introduced by the model. The findings provide empirical data on the limitations of current LLMs for real-time control and highlight the necessity of verification loops in AI-assisted networking.

2. Literature Review

The systematic review carried out for this project analysed 24 works published between 2020 and 2025 that address SDN and IDN architectures, security mechanisms, automation frameworks, and the use of AI or LLM-based techniques in network management [Herbaut et al. 2021, Ahmad et al. 2023, El-Rif et al. 2023, Alonso-Lopez et al. 2022, Minhas et al. 2024b, Lin et al. 2025]. More information on this systematic literature review can be found in the full text review³. The selection followed a protocol inspired by PRISMA, with explicit inclusion and exclusion criteria and full-text assessment of conceptual and methodological clarity. The studies can be grouped in three main lines: SDN-based security with automation, intent-driven orchestration frameworks, and AI or LLM support for configuration and diagnosis.

A first group of works focuses on SDN as an enabler for dynamic security. In these studies, the controller acts as a central point that observes network traffic and installs mitigation rules at run time [Ahmad et al. 2023, Dungarani and Gujjar 2024]. Typical examples include DDoS detection and mitigation pipelines, anomaly detection based on flow-level statistics, and context-aware enforcement of access policies [Abassi and Tabbane 2023, Zhang et al. 2025]. The results confirm that SDN programmability can support automatic updates of forwarding rules and security policies

¹<https://opennetworking.org/onos/>

²<https://mininet.org/>

³<https://github.com/rideckszz/onos-nl-intent-mvp/blob/master/RSL-LLM-SDN.pdf>

when new events occur. However, the translation from high-level security objectives to concrete flow rules or policies is still encoded manually in controller applications. The operator needs to understand the internal abstractions of each framework and map operational requirements to rules that the controller can enforce.

A second line of work extends this idea with intent-based abstractions. Intent-Driven Networking models introduce objects that represent desired outcomes, such as connectivity between specific endpoints with selected constraints or guarantees [Herbaut et al. 2021, Alonso-Lupez et al. 2022]. In ONOS, intents such as *HostTo-HostIntent* capture the requirement that two hosts must be connected, while the controller is responsible for choosing paths and handling failures. Studies on ONOS and other IDN frameworks show that intents can simplify policy management, support multi-layer or multi-domain scenarios, and provide monitoring feedback when an intent is violated or cannot be satisfied [Hyder and Fatima 2021, El-Rif et al. 2023, Wang et al. 2025]. Even in these settings, the vocabulary of intents is finite and pre-defined. Operators or applications must still instantiate intent types by providing parameters such as host identifiers, priorities, and constraints. The literature reports limited support for translating informal objectives into these formal objects or for validating that the resulting configuration matches the original goal [Alonso-Lupez et al. 2022, de Trizio et al. 2024].

Within this context, the review identifies an explicit gap in the translation process. Existing SDN and IDN solutions provide mechanisms to express policies as intents, but they do not define a general mediation layer that takes human-level descriptions of network goals and produces validated controller artefacts. Security properties often appear as attributes or constraints attached to specific intents instead of first-class elements with dedicated reasoning and verification [Abassi and Tabbane 2023, Alonso-Lupez et al. 2022]. Only a few works discuss conflict detection, policy consistency, or formal checks that confirm alignment between declared security objectives and the installed configuration [de Trizio et al. 2024]. This gap motivates the design of a mediator that can interpret natural-language requests, map them to well-typed intent operations, and check controller state after each change.

A third group of studies explores natural-language and AI-based support for network management. Some works propose assistants that help operators interact with documentation, generate configuration templates, or interpret logs [Minhas et al. 2024b, Lingga et al. 2025]. Others investigate machine learning models, including LLMs, to classify traffic, detect attacks, or recommend responses in SDN environments [Kou et al. 2025, Parajuli et al. 2025]. These efforts indicate that models trained on textual and code data can bridge part of the gap between informal descriptions and low-level artefacts. They also show that natural-language interfaces can reduce the cognitive effort required to remember specific commands or configuration syntax. However, most proposals keep a human in the loop as the final decision-maker and stop at generating suggestions or scripts that need manual review. The integration with SDN and IDN controllers remains loose, without an explicit architecture in which natural-language requests are systematically converted into controller intents with verification steps.

The reviewed literature also highlights risks associated with placing LLMs near the control plane. Studies on LLM robustness and security demonstrate that models are vulnerable to prompt injection, adversarial inputs, and inconsistent behaviour across

queries [Lin et al. 2025, Abouelseoud and El-Aasser 2025]. In security-oriented workflows, these properties raise concerns about determinism, repeatability, and traceability of decisions [Rivera et al. 2020, Minhas et al. 2024b]. If an LLM is allowed to send configuration commands directly to a controller, a malicious or ambiguous prompt could lead to incorrect policies or unintended connectivity. The literature therefore recommends safeguards such as constrained output formats, audit logs, static checks, and human or programmatic validation before enforcement.

From these three lines, two conclusions are relevant for this project. First, SDN and IDN frameworks already provide the mechanisms required to express connectivity and security policies as intents and to apply them programmatically [Hyder and Fatima 2021, El-Rif et al. 2023]. Second, there is still no end-to-end architecture that connects natural-language descriptions of network goals to these intent frameworks with explicit validation and measurement of behaviour. Existing work on LLMs in networking suggests that models can support interpretation and code generation, but does not quantify their performance when integrated into the control loop of a concrete controller such as ONOS [Minhas et al. 2024b, Lingga et al. 2025].

The mediator proposed in this study is positioned at this interface. It adopts the ONOS intent framework as the target representation, uses a constrained JSON schema to capture the output of the language model, and performs verification by querying the controller state after each operation. The experimental methodology builds on performance evaluation practices used in SDN studies [Jain 1991, Gupta et al. 2022] and on the systematic review findings about security, intents, and LLM-based mediation. In this way, the project advances from the conceptual discussion of LLM-assisted intent translation to an implementation that can be measured in terms of translation accuracy, latency, and consistency of the resulting controller state.

3. Objectives

3.1. General Objective

Investigate the feasibility of using a Large Language Model (LLM) as a mediator for the configuration of Software-Defined Networks (SDN) and Intent-Driven Networks (IDN), evaluating its ability to interpret natural-language requests and translate them into ONOS intents with measurable accuracy, response time, and configuration consistency.

3.2. Specific Objectives

- Develop a functional mediator capable of bridging the semantic gap between natural-language inputs and the structured ONOS REST API using a Large Language Model.
- Implement a verification mechanism within the mediation loop to ensure that the state of the network controller accurately reflects the requested operations.
- Evaluate the correctness of the translation process by measuring success rates across diverse intent operations (connection, listing, deletion) and network topologies.
- Decompose the total configuration latency into planning (LLM inference) and execution (controller interaction) components to identify performance bottlenecks.

- Assess the scalability of the mediator by analyzing how response latency and reliability evolve as the complexity and size of the network topology increase.
- Compare the performance of the mediator against a baseline of direct API interaction to quantify the trade-off between the ease of natural-language configuration and execution time.
- Document the experimental procedures, mediator architecture, and observed limitations to support reproducibility and future research in AI-assisted networking.

4. Methodology

The project adopts an experimental methodology with the objective of developing and evaluating a mediator that uses a large language model to interpret natural language instructions and translate them into network intents. The approach follows structured guidelines from performance evaluation studies [Jain 1991], so that each step is clearly defined, executed, and analysed. The goal is to verify if the mediator can act as a correct interface for automating configurations in a Software Defined Network controlled by ONOS [Kreutz et al. 2015].

4.1. System Architecture

The system is organised in a client-server architecture. The ONOS controller maintains the global network state and applies intents that define connectivity policies [Kreutz et al. 2015]. Mininet emulates the data plane and creates virtual switches and hosts in a reproducible way [Kaur et al. 2014]. Instead of a single topology, the experiments use a set of small synthetic topologies generated in Mininet: linear chains with 3, 6, and 9 hosts, and two-level trees with fanout 2 and 3. These topologies represent networks with increasing numbers of hosts and different path structures, which is useful to observe how the mediator behaves as the size of the emulated network grows.

The mediator operates as a separate software component that interacts with ONOS through its REST API. It receives textual commands in natural language, sends each instruction to a planning component based on a large language model, and receives a structured JSON plan as a response. The plan describes the operation to execute (for example, connecting two hosts, listing intents, deleting intents between hosts, or deleting all intents) and the parameters involved, such as host identifiers and priority. The mediator validates this JSON object, resolves host identifiers to the internal format used by ONOS, and constructs the corresponding REST requests. After execution, it queries ONOS again to verify that the controller state matches the requested operation.

4.2. Experimental Testbed

The experimental environment was deployed on a physical workstation running Linux (Ubuntu 22.04 LTS). The host machine was equipped with an Intel Core i7-7700 processor (4 cores, 8 threads) and 32 GB of DDR4 RAM. This hardware configuration provided sufficient resources to run the Mininet emulator, the ONOS controller (version 2.5.0), and the mediator logic concurrently without resource contention affecting the latency measurements.

The planning component utilised the `gemma3:12b-vision` model, accessed via the Unicamp Institute of Computation LLM API. This model was selected for its

balance between inference speed and reasoning capability, making it suitable for near-real-time control tasks. The mediator software was implemented in Python 3.12, using the `requests` library for API interaction and standard logging modules for metric collection. Additional libraries and code details can be found at the GitHub repository for the project⁴.

4.3. Development and Configuration

The development process began with a review of the theoretical foundations of Software Defined Networking, network programmability, and the intent abstraction model [Kreutz et al. 2015]. This phase established the conceptual basis for understanding how intents operate in ONOS and what constraints exist for their automated generation. It also defined the evaluation objectives, the metrics to be collected, and the expected outcomes of each experiment.

After defining the scope, the environment was prepared by installing ONOS and Mininet. The controller was configured to expose its REST interface, and Mininet was configured to use a remote controller at the IP and port where ONOS was running. Each experiment started by instantiating one of the predefined topologies in Mininet and attaching it to ONOS. A `pingAll` command was then executed to populate MAC tables and ARP entries and to confirm that the basic IP connectivity in the emulated topology was working.

Once connectivity was validated, the mediator queried ONOS repeatedly through `GET /hosts` until the controller reported at least two hosts. The mediator mapped each host to a logical token (*h1*, *h2*, and so on) based on its IP address or MAC address. A short warm-up phase was then executed. In this phase, the mediator installed a fixed number of *HostToHostIntent* instances between randomly selected pairs of hosts. The warm-up avoided starting measurements with an empty intent table and forced ONOS to exercise its internal path computation and state management before the actual measurements began.

4.4. Mediator Operations

For each natural language instruction, the mediator calls the planner with a fixed system prompt that specifies the JSON schema to be produced. The planner returns a JSON object that contains at least the *operation* field and, depending on the case, a list of hosts (*"hosts"*), a source host (*"src_host"*), a destination host (*"dst_host"*), and a priority value. The mediator removes possible formatting artefacts such as Markdown code fences, parses the JSON, and validates the presence of the required fields.

Four classes of operations are supported in the experiments:

- **Connect Hosts:** Creates a *HostToHostIntent* between two hosts. The mediator uses prompts of the form *"connect hX and hY"*, resolves the tokens to ONOS IDs, and sends a `POST` request.
- **List Intents:** Corresponds to a `GET /intents` request, triggered by prompts like *"list intents"*. It measures read operation latency.
- **Specific Deletion:** Removes intents involving a subset of hosts. Supported forms include *"remove intents from hX and hY"* (pair deletion) and *"remove intents from*

⁴<https://github.com/rideckszz/onos-nl-intent-mvp>

hX” (single host deletion). The mediator identifies matching intents and sends DELETE requests.

- **Delete All:** Removes all intents associated with the mediator’s application ID, triggered by *”remove all intents”*.

4.5. Evaluation Metrics and Procedure

The main quantitative evaluation was performed with an automated scaling script. For each topology, the script executed the following sequence: (i) start Mininet and attach to ONOS, (ii) run `pingAll` for host discovery, (iii) run the warm-up phase, and (iv) execute a fixed number of natural language instructions for each operation class.

Latency was decomposed into three components:

- **Planning Time:** Measured from the prompt transmission to the LLM until the reception of the JSON plan.
- **Controller Time:** Measured from the first REST API request to the final response from ONOS.
- **Verification Time:** Measured during the subsequent queries used to confirm the state change.

Correctness was evaluated through success indicators derived from the controller state (e.g., HTTP 200 OK status and the presence/absence of specific intent IDs). The results were compared with equivalent configurations applied manually through the ONOS REST API, which served as a baseline.

To summarise the steps, Table 1 presents an overview of the development and experimentation phases and their main activities.

Table 1. Overview of Development and Experimentation Phases

Phase	Main Activities
Theoretical Review	Study of SDN and IDN concepts; analysis of the ONOS intent framework; definition of objectives, metrics, and scope.
Environment Setup	Installation and configuration of ONOS and Mininet; configuration of the remote controller; deployment of linear and tree topologies; validation of connectivity and basic intent operations.
Mediator Implementation	Development of the Python-based mediator; integration with the LLM planner; construction of REST requests; implementation of verification functions and logging.
Experiment Design	Definition of natural language scenarios for connect, list, delete between hosts, and delete all intents; planning of repeated trials per topology; configuration of the CSV logging format.
Execution and Data Collection	Automated execution of instructions across multiple topologies; monitoring of planner behaviour and controller responses; collection of metrics including success rate and latency components.
Analysis and Validation	Aggregation of results per operation and topology; comparison with manual REST configurations; interpretation of limitations and identification of improvement opportunities.
Documentation	Compilation of procedures, results, and analysis into a structured report; maintenance of the public repository with code and experiment artefacts.

5. Results

This section presents the quantitative data collected during the experiments. The evaluation focuses on success rates, latency decomposition, scalability trends, and error distribution. The metrics follow the performance evaluation guidelines established in the methodology.

5.1. Success Rates

The mediator demonstrated a high capacity for correctly interpreting and applying intents. As shown in Figure 1, the system achieved a success rate above 0.85 across all five topologies, with linear topologies showing slightly higher consistency. Figure 2 details performance by operation; *Host Connection* and *List Intents* achieved 100% success, while *Specific Deletion* proved more challenging (74%) due to the complexity of resolving partial deletion contexts.

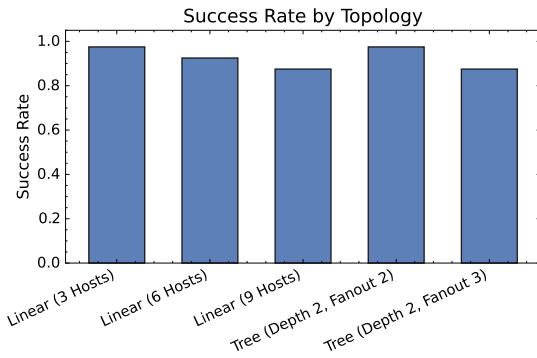


Figure 1. Success rate by topology.

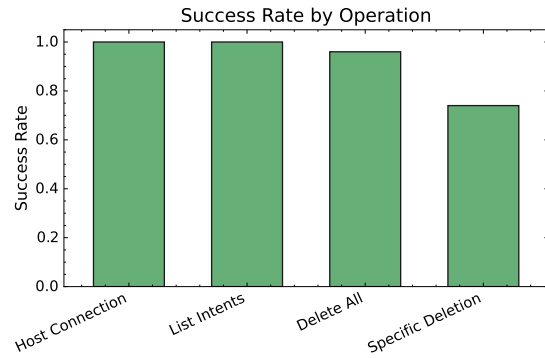


Figure 2. Success rate by operation type.

5.2. Latency Analysis

Latency was decomposed to distinguish the cost of the Large Language Model from the network controller. Figure 3 and Figure 4 illustrate this breakdown. The planning phase (LLM inference) dominates the total time, averaging approximately 50 seconds regardless of topology size. In contrast, the system execution time (Controller + Verification) remains negligible, operating consistently in the 10ms to 25ms range. This confirms that the REST API interaction introduces minimal overhead.

Regarding scalability, Figure 5 shows the trend for linear topologies. The total latency remains effectively constant as the number of hosts increases from 3 to 9. The observed standard deviation reflects variability in the model’s token generation speed rather than network congestion, indicating that prompt complexity for these scenarios did not significantly impact inference time.

5.3. Error Distribution

Failures observed during the experiments were categorized to understand the mediator’s limitations (Figure 6). The most frequent issue was *Verification Error*, where the mediator successfully sent a request, but the post-execution check revealed the intent was not correctly applied. *Incomplete Deletion* occurred in rare cases where complex deletion criteria were not fully met.

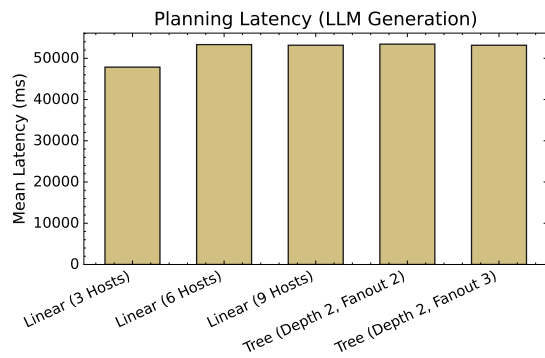


Figure 3. Planning latency (LLM inference).

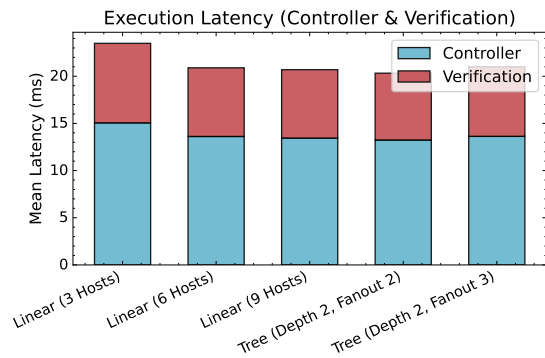


Figure 4. Execution latency (System).

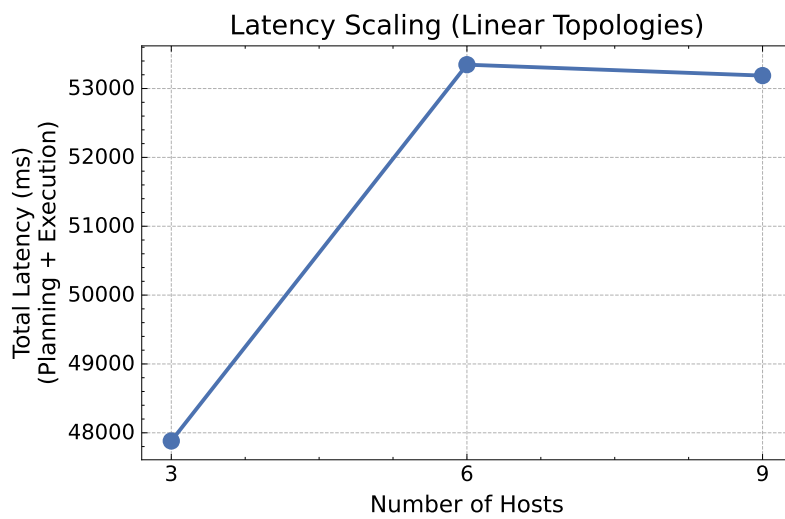


Figure 5. Latency scaling trend in linear topologies.

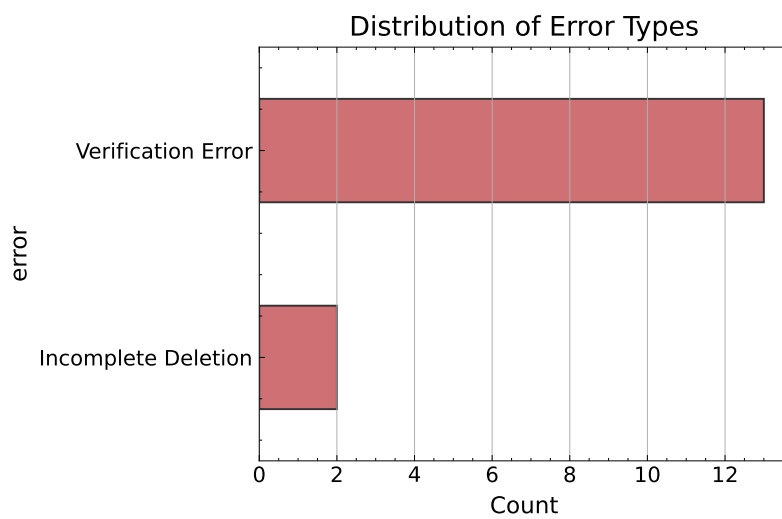


Figure 6. Distribution of observed error types.

6. Discussion

The experimental results validate the feasibility of using an LLM as a translation layer for intent-based networking, while highlighting significant performance trade-offs. The high success rates for connection and list operations confirm that the mediator can effectively bridge the semantic gap between natural language and the ONOS intent framework [Kreutz et al. 2015]. However, the performance metrics reveal distinct limitations regarding latency and complex state management.

6.1. Interpretation of Latency and Scalability

The latency breakdown indicates a clear bottleneck in the planning phase. The mediator requires approximately 50 seconds to generate a valid JSON plan. In contrast, the controller execution time remains in the order of milliseconds. This disparity restricts the applicability of the current architecture to configuration management tasks where human-scale latency is acceptable. The system is not viable for reactive scenarios, such as dynamic failure recovery or real-time security mitigation, which require response times in the sub-second range [Gupta et al. 2022].

The scaling trend observed in the linear topologies suggests that the inference time is relatively independent of the network size for small inputs. The prompt complexity did not grow linearly with the number of hosts, resulting in a flat latency curve. This implies that the cost of mediation is fixed per instruction rather than per network node, at least for the scale of topologies tested.

6.2. Reliability and Error Analysis

The drop in success rate for the *Specific Deletion* operation (74%) highlights a challenge in context management. Unlike creation requests, which are stateless, deletion requests require the mediator to understand the current state of the network to identify which intents to remove. The *Verification Error* type suggests that the LLM occasionally generated syntactically correct requests that semantically failed to target the existing intents. This confirms concerns raised in the literature regarding the probabilistic nature of generative models in deterministic control loops [Minhas et al. 2024b].

6.3. Comparison with Traditional Automation

Compared to manual interactions with the REST API, the mediator introduces a significant time overhead but reduces the cognitive load required to recall API syntax. For standard operations, the reliability is comparable to manual configuration. However, the lack of deterministic guarantees in the *Specific Deletion* scenario indicates that the mediator requires a robust verification loop to be operationally safe. The inclusion of a post-execution check in the methodology proved essential for detecting these inconsistencies [Jain 1991].

The results suggest that while LLMs can act as cognitive interfaces for IDN, they should be decoupled from the real-time control plane. They function best as an asynchronous orchestration layer that validates user intent before interacting with the controller.

7. Conclusion

This work evaluated the use of Large Language Models as a mediation layer for Intent-Driven Networking in ONOS. The development of a functional mediator and the execution of controlled experiments in a Mininet environment provided quantitative evidence regarding the potential and limitations of this approach.

The results confirm that LLMs can accurately translate natural language instructions into structured ONOS intents, achieving 100% success rates for basic connectivity and listing operations. This demonstrates the feasibility of using generative models to lower the barrier of entry for network configuration, abstracting the complexity of the underlying API. However, the analysis of specific deletion operations revealed reliability issues in state-dependent tasks, emphasising the need for rigorous verification mechanisms in the loop.

Performance analysis identified the planning phase as the primary bottleneck, with inference times largely exceeding the execution capabilities of the SDN controller. While the controller and verification steps operate in the millisecond range, the LLM inference introduces a latency of approximately 50 seconds per request. This characteristic limits the use of such mediators to non-real-time orchestration and management tasks where ease of use is prioritized over speed.

Future work should investigate methods to reduce planning latency, such as fine-tuning smaller models for the specific domain of network intents or employing hybrid architectures that combine LLM interpretation with deterministic rule engines. Additionally, expanding the scope to cover more complex intent types and larger topologies will be necessary to further validate the scalability of the approach in production-grade environments.

7.1. Limitations and Future Work

The limitations of this work are mainly related to the scope of the experimental evaluation and to the characteristics of the available ONOS REST API. The experiments were designed to validate the complete set of intent operations exposed by the controller interface and to analyse their behaviour under natural language mediation. Within this scope, all supported intent forms were exercised.

The evaluation was conducted using Mininet with linear and tree topologies of limited size. These topologies are sufficient to test correctness, verification logic, and latency decomposition, but they do not represent large-scale or highly dynamic production networks. As a result, the reported metrics should be interpreted as representative of controlled environments rather than as absolute bounds for operational deployments.

Correctness was evaluated through controller state inspection and verification queries after each operation. This approach confirms that the intended configuration was applied at the control plane level. However, the evaluation does not include systematic data plane measurements after each intent installation or removal. Future evaluations could extend verification with automated end-to-end connectivity checks or path validation to further characterise the effects of translated intents.

The main performance limitation observed is the latency of the planning stage. The inference time dominates the total execution time and restricts the use of the me-

diator to non-real-time configuration tasks. In contrast, the controller interaction and verification steps consistently operate at millisecond scale, indicating that the overhead is not introduced by the SDN control plane.

Future work will focus on reducing planning latency and expanding the evaluation scenarios. This includes testing alternative models with lower inference cost, evaluating larger and more dynamic topologies, and extending verification to include data plane observations. These steps are required to assess the applicability of the approach in operational environments beyond the controlled settings used in this study.

References

- Abassi, Y. and Tabbane, N. (2023). Context aware life cycle for intent-based networking systems. In *2023 IEEE Tenth International Conference on Communications and Networking (ComNet)*, pages 1–8. ISSN: 2473-7585.
- Abouelseoud, Y. and El-Aasser, M. (2025). Practical implementation of intent-based networking using OpenDaylight and rasa. In *2025 International Conference on Machine Intelligence and Smart Innovation (ICMISI)*, pages 100–105.
- Ahmad, I., Malinen, J., Christou, F., Porambage, P., Kirstädter, A., and Suomalainen, J. (2023). Security in intent-based networking: Challenges and solutions. In *2023 IEEE Conference on Standards for Communications and Networking (CSCN)*, pages 296–301. ISSN: 2644-3252.
- Alonso-Lupez, J. A., Hernández, L. A. M., Arteaga, S. P., Orozco, A. L. S., Villalba, L. J. G., Pastor, A., and García, D. L. (2022). Level of trust and privacy management in 6g intent-based networks for vertical scenarios. In *2022 1st International Conference on 6G Networking (6GNet)*, pages 1–4.
- de Trizio, F., Sciddurlo, G., Rutigliano, D., Piro, G., and Boggia, G. (2024). A novel malicious intent detection approach in intent-based enterprise networks. In *2024 20th International Conference on Network and Service Management (CNSM)*, pages 1–7. ISSN: 2165-963X.
- Dungarani, R. and Gujjar, S. N. (2024). SDN security: Taming the wild west of network automation. In *2024 IEEE International Conference on Blockchain and Distributed Systems Security (ICBDS)*, pages 1–13.
- El-Rif, E., Leivadeas, A., and Falkner, M. (2023). Intent expression through natural language processing in an enterprise network. In *2023 IEEE 24th International Conference on High Performance Switching and Routing (HPSR)*, pages 1–6. ISSN: 2325-5609.
- Gupta, N., Maashi, M. S., Tanwar, S., Badotra, S., Aljebreen, M., and Bharany, S. (2022). A comparative study of software defined networking controllers using mininet. *Electronics*, 11(17):2715.
- Herbaut, N., Correa, C., Robin, J., and Mazo, R. (2021). SDN intent-based conformance checking: application to security policies. In *2021 IEEE 7th International Conference on Network Softwarization (NetSoft)*, pages 181–185. ISSN: 2693-9789.

- Hyder, M. F. and Fatima, T. (2021). Towards crossfire distributed denial of service attack protection using intent-based moving target defense over software-defined networking. *IEEE Access*, 9:112792–112804.
- Jain, R. (1991). *The Art of Computer Systems Performance Analysis: Techniques For Experimental Design, Measurement, Simulation, and Modeling*, NY: Wiley. Wiley.
- Kaur, K., Singh, J., and Ghumman, N. S. (2014). Mininet as software defined networking testing platform. In *Proceedings of the International Conference on Communication, Computing & Systems (ICCCS)*, pages 139–142.
- Kou, S., Yang, C., and Gurusamy, M. (2025). SAFLA: Semantic-aware full lifecycle assurance for intent-driven networks. *IEEE Transactions on Cognitive Communications and Networking*, pages 1–1.
- Kreutz, D., Ramos, F. M. V., Veríssimo, P. E., Rothenberg, C. E., Azodolmolky, S., and Uhlig, S. (2015). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1):14–76.
- Lin, C., Huang, Q., and Lin, Y. (2025). SentiNet: Semantic-driven intent networking for SDN. In *2025 IEEE 7th International Conference on Communications, Information System and Computer Engineering (CISCE)*, pages 102–108. ISSN: 2833-2423.
- Lingga, P., Jeong, J., and Dunbar, L. (2025). ICSC: Intent-based closed-loop security control system for cloud-based security services. *IEEE Communications Magazine*, 63(4):169–175.
- Minhas, S., Jaswal, R., Sharma, A., and Singla, S. (2024a). Revolutionizing networking: A comprehensive overview of intent-based networking. In *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNO-COMP)*, pages 463–468. IEEE.
- Minhas, S., Jaswal, R., Sharma, A., and Singla, S. (2024b). Revolutionizing networking: A comprehensive overview of intent-based networking. In *2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*, pages 463–468.
- Parajuli, A., Sivalingam, K. M., and Raj Kanagarathinam, M. (2025). Leveraging large language models for intent-based networking: A framework for automated configuration and optimization. In *2025 IEEE International Conference on Communications Workshops (ICC Workshops)*, pages 911–916. ISSN: 2694-2941.
- Rivera, S., Fei, Z., and Griffioen, J. (2020). POLANCO: Enforcing natural language network policies. In *2020 29th International Conference on Computer Communications and Networks (ICCCN)*, pages 1–9. ISSN: 2637-9430.
- Wang, Z., Lin, S., Yan, G., Ghorbani, S., Yu, M., Zhou, J., Hu, N., Baruah, L., Peters, S., Kamath, S., Yang, J., and Zhang, Y. (2025). Intent-driven network management with multi-agent LLMs. In *Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework*, ACM Conferences, pages 347–362. Proceedings of the ACM SIGCOMM 2025 Conference.

Zhang, T., Yang, H., Shen, J., Pandi, V., Arya, V., and Gupta, B. B. (2025). ABACPR: Attribute-based access control supporting policy reconstruction for intent-based networking. *IEEE Internet of Things Journal*, pages 1–1.